Marketing Campaign Performance Insights – Python Data Analysis Project

Step 1: Import the required libraries

Open Jupyter Notebook and create a new notebook named Marketing_Campaign_Performance_Insights.ipynb.

Run the following code in the first cell:

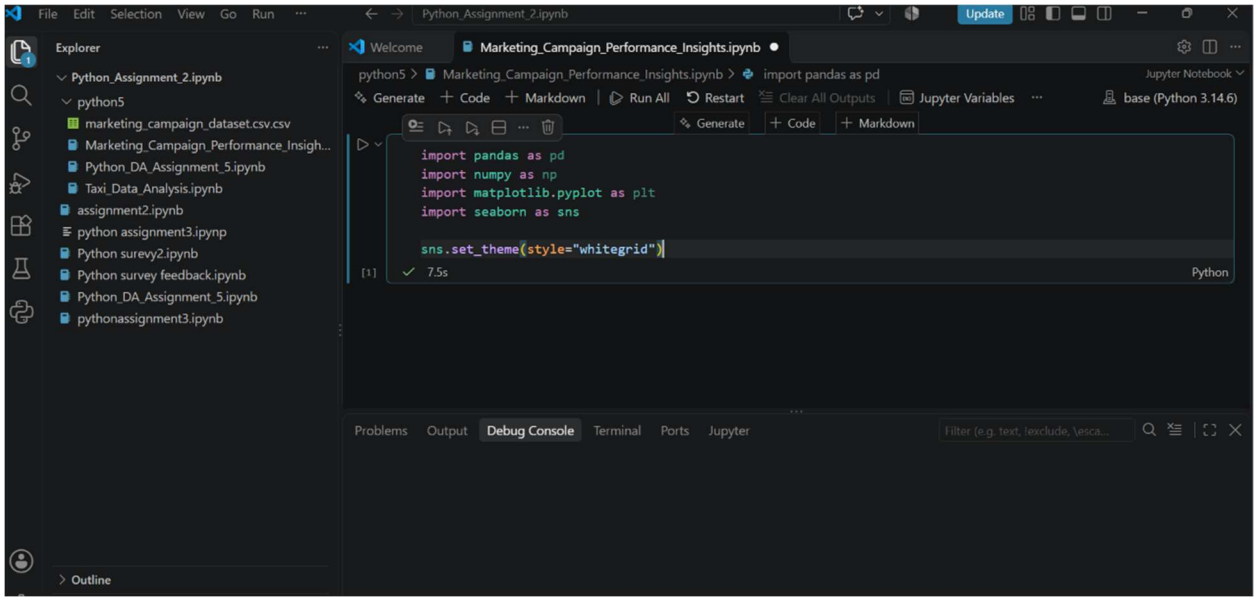
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

sns.set_theme(style="whitegrid")

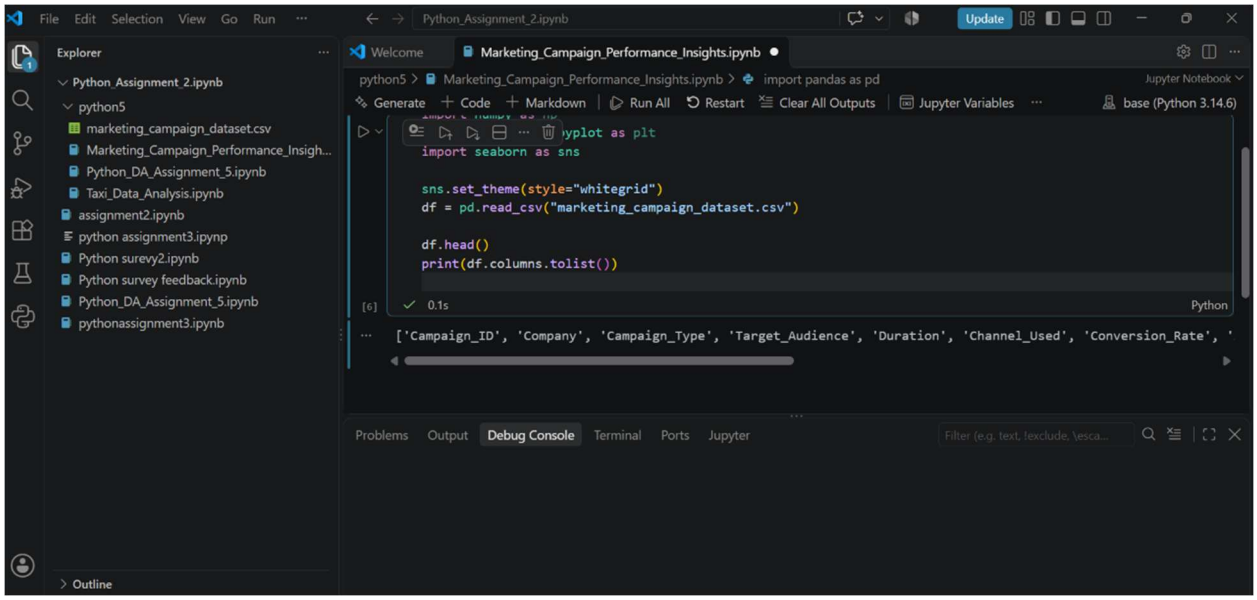


Explanation:

- pandas: Loads, cleans, and analyzes data.
- numpy: Performs numerical calculations.
- matplotlib: Creates charts.
- seaborn: Creates statistical visualizations.

- # Step 2: Load the dataset

- Place your CSV file in the same folder as your Jupyter Notebook
- df = pd.read_csv("marketing_campaign_dataset.csv")df.head()
- Replace the filename if CSV has a different name.
- **Check column names**
- print(df.columns.tolist())

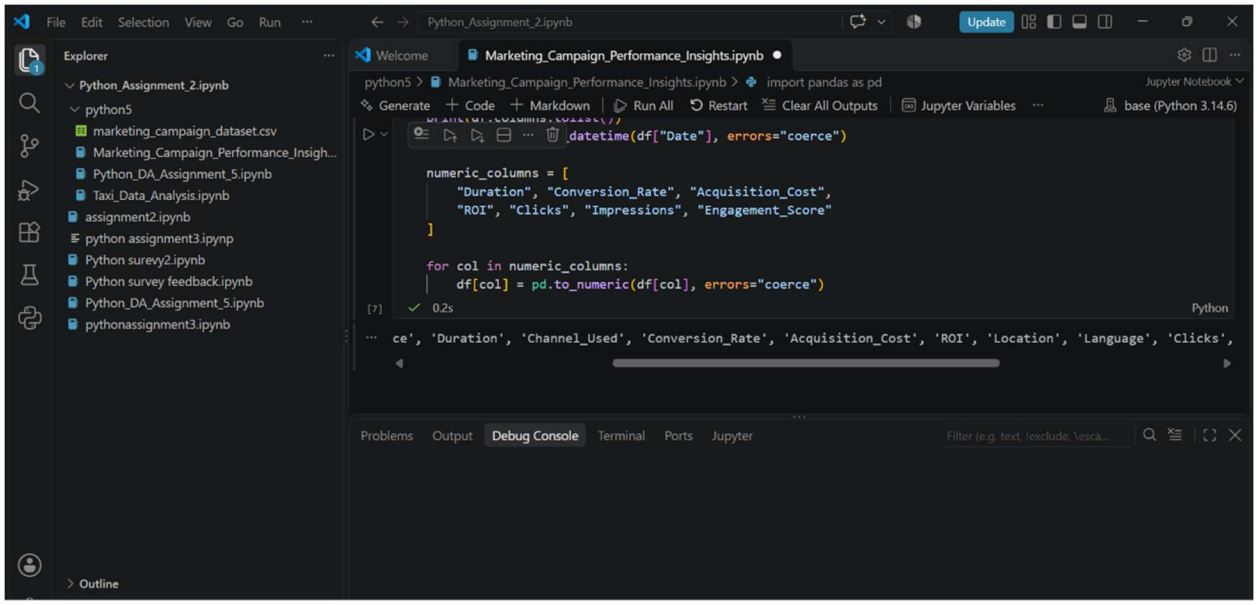


Convert columns to appropriate data types

```
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

numeric_columns = [
    "Duration", "Conversion_Rate", "Acquisition_Cost",
    "ROI", "Clicks", "Impressions", "Engagement_Score"
]

for col in numeric_columns:
    df[col] = pd.to_numeric(df[col], errors="coerce")
```

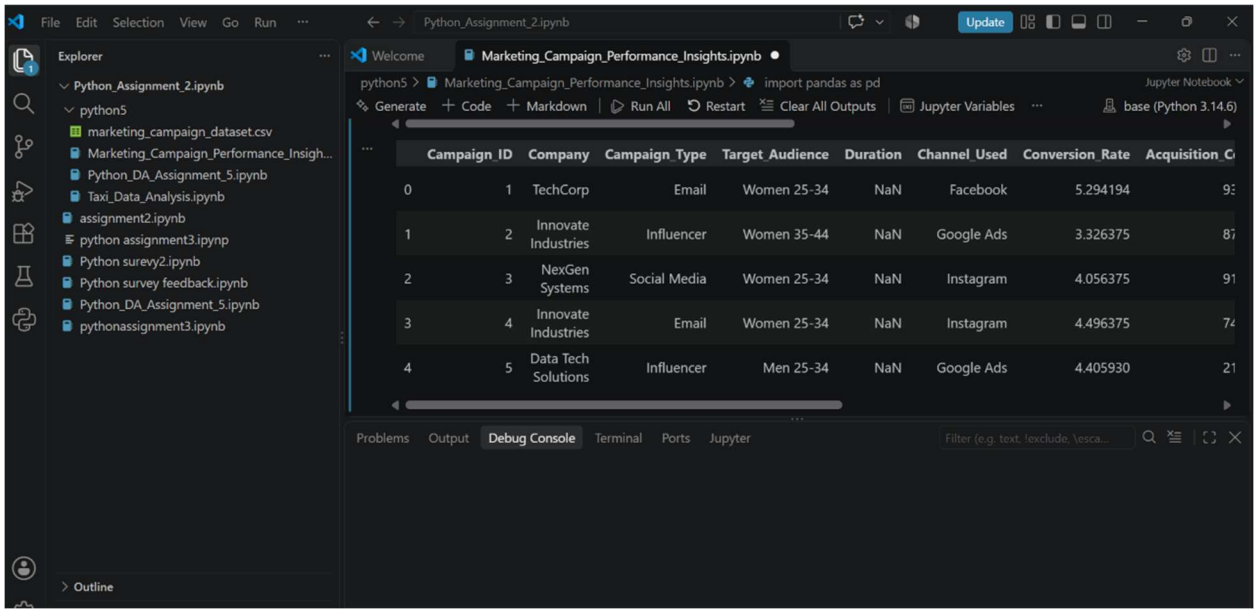


Conversion_Rate, Acquisition_Cost, or ROI contain percentage signs, currency symbols, or other formatting, these must be cleaned before numeric conversion.

Part 1: Descriptive Analysis

Task 1: Display the first few rows

df.head()

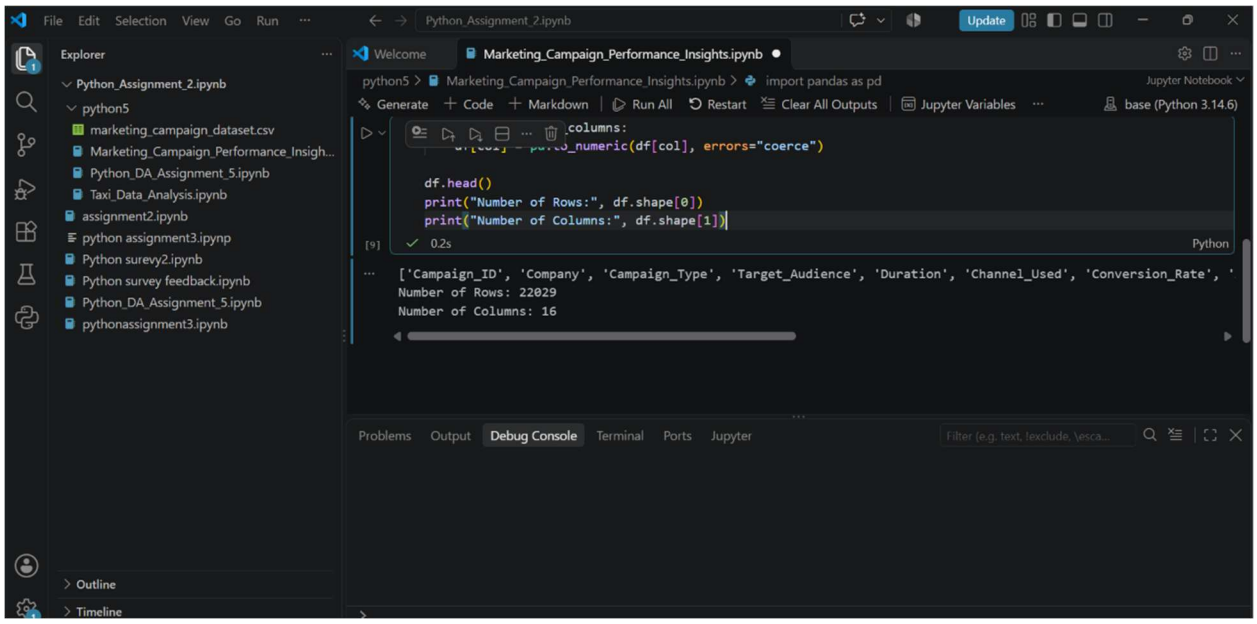


	Campaign_ID	Company	Campaign_Type	Target_Audience	Duration	Channel_Used	Conversion_Rate	Acquisition_C
0	1	TechCorp	Email	Women 25-34	NaN	Facebook	5.294194	9
1	2	Innovate Industries	Influencer	Women 35-44	NaN	Google Ads	3.326375	8
2	3	NexGen Systems	Social Media	Women 25-34	NaN	Instagram	4.056375	91
3	4	Innovate Industries	Email	Women 25-34	NaN	Instagram	4.496375	7
4	5	Data Tech Solutions	Influencer	Men 25-34	NaN	Google Ads	4.405930	21

Task 2: Find the number of rows and columns

print("Number of Rows:", df.shape[0])

print("Number of Columns:", df.shape[1])



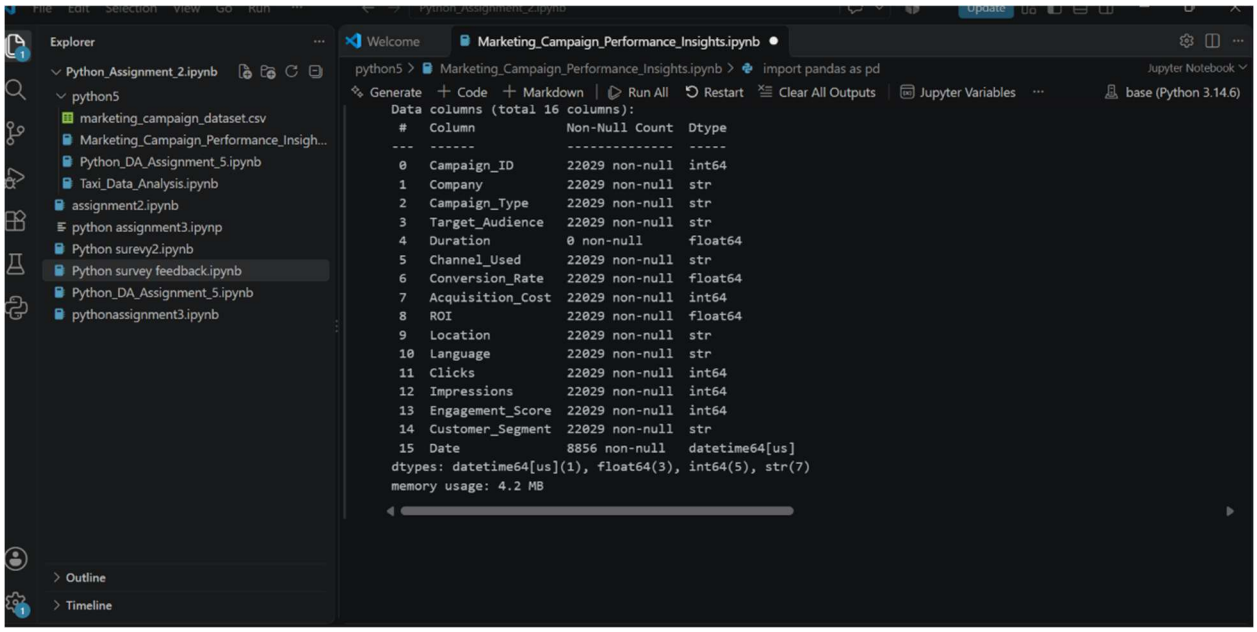
Explanation:

- shape[0] gives the number of rows.
- shape[1] gives the number of columns.

Task 3: Display dataset information

df.info(),

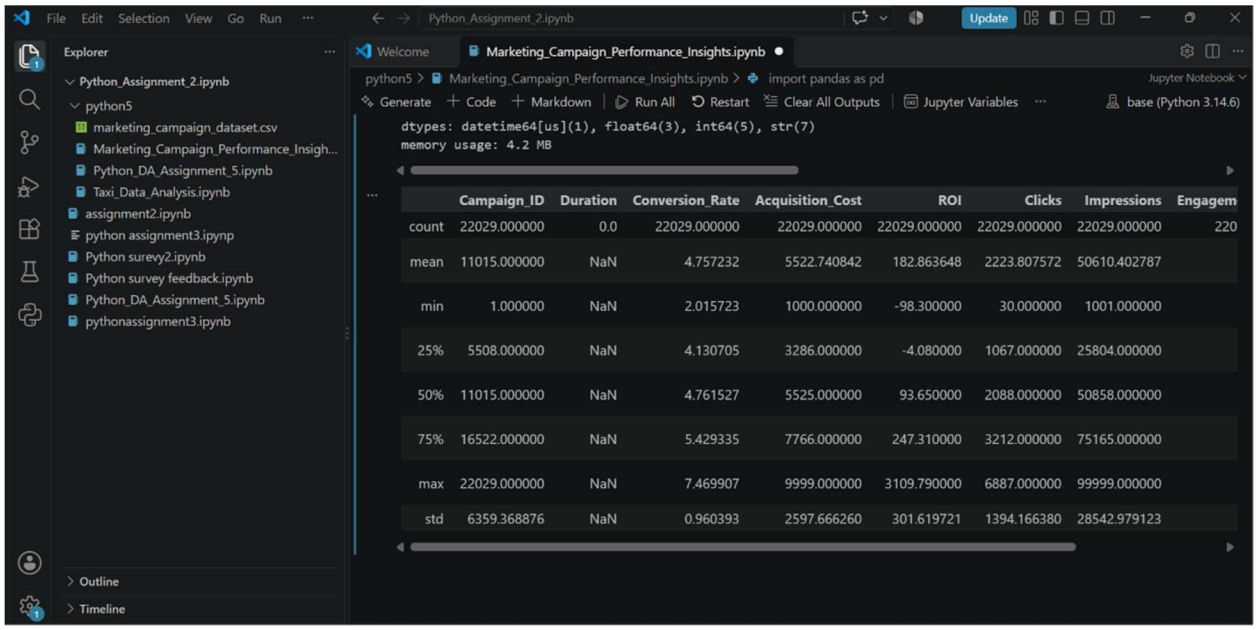
This displays column names, data types, and non-null counts.



Task 4: Generate descriptive statistics

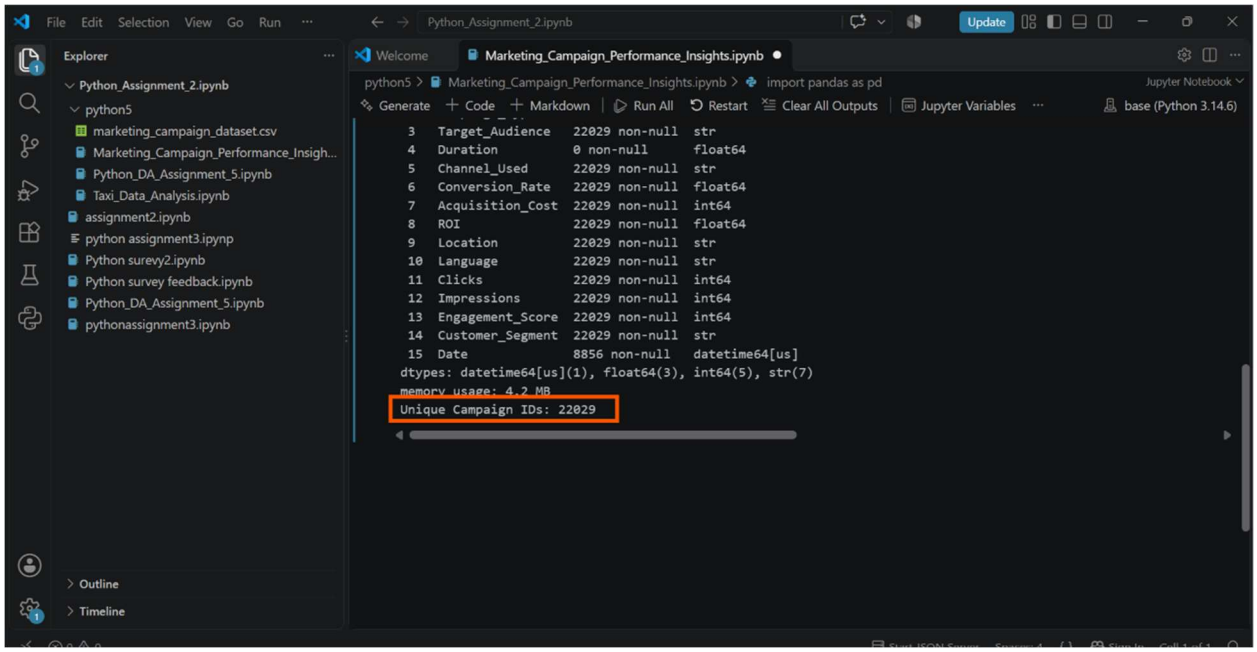
df.describe()

This calculates count, mean, standard deviation, minimum, maximum, and quartiles for numerical columns.



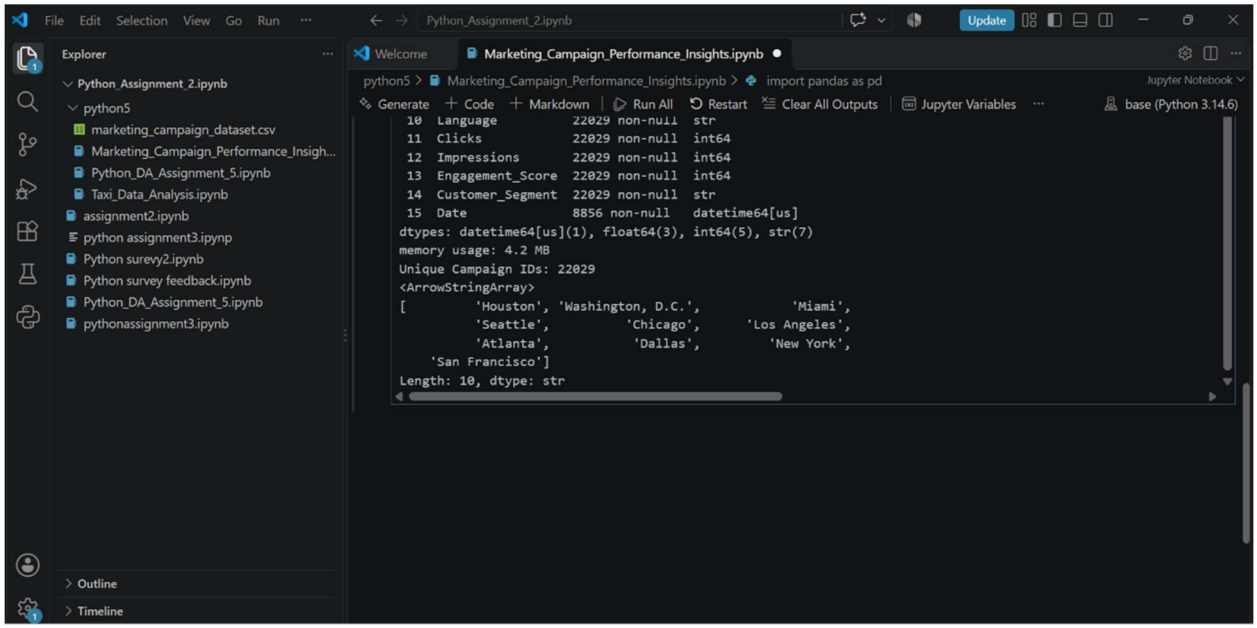
Task 5: Count unique Campaign IDs

print("Unique Campaign IDs:", df["Campaign_ID"].nunique())



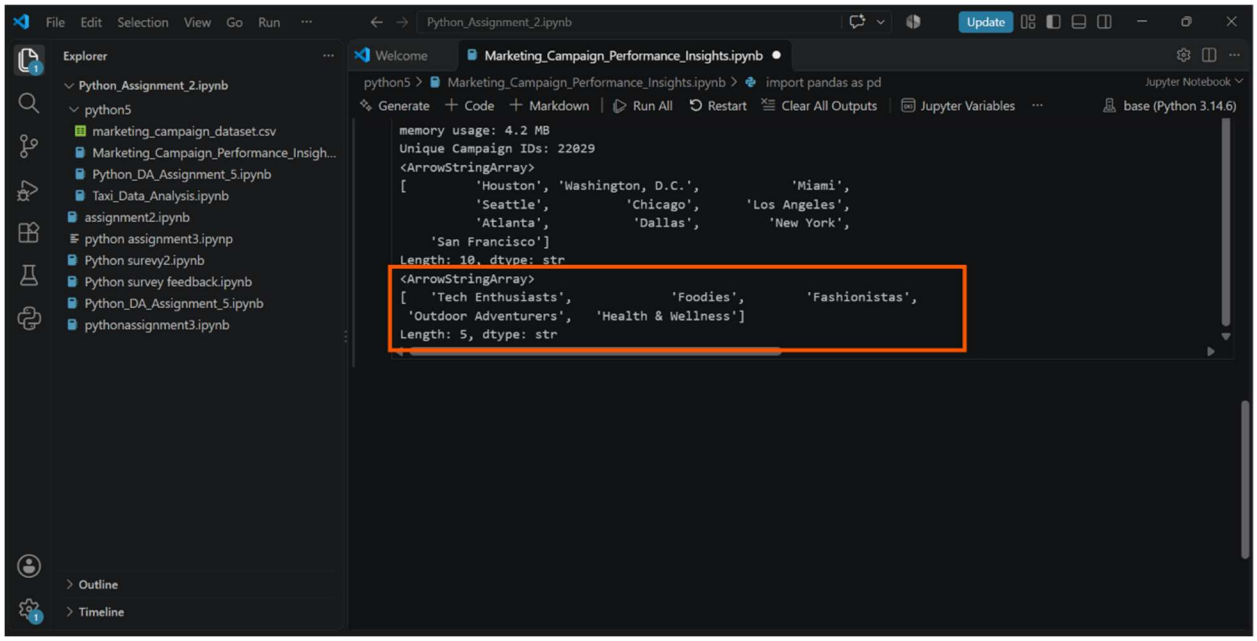
Task 6: Display unique locations

print(df["Location"].unique())



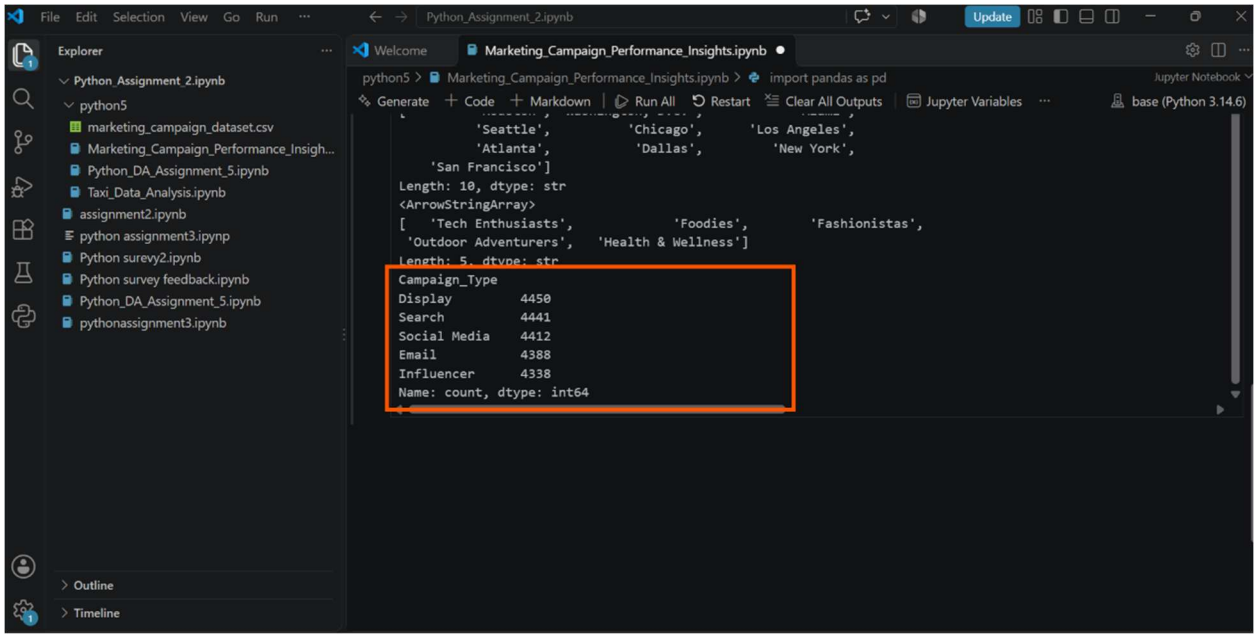
Task 7: Display unique customer segments

print(df["Customer_Segment"].unique())



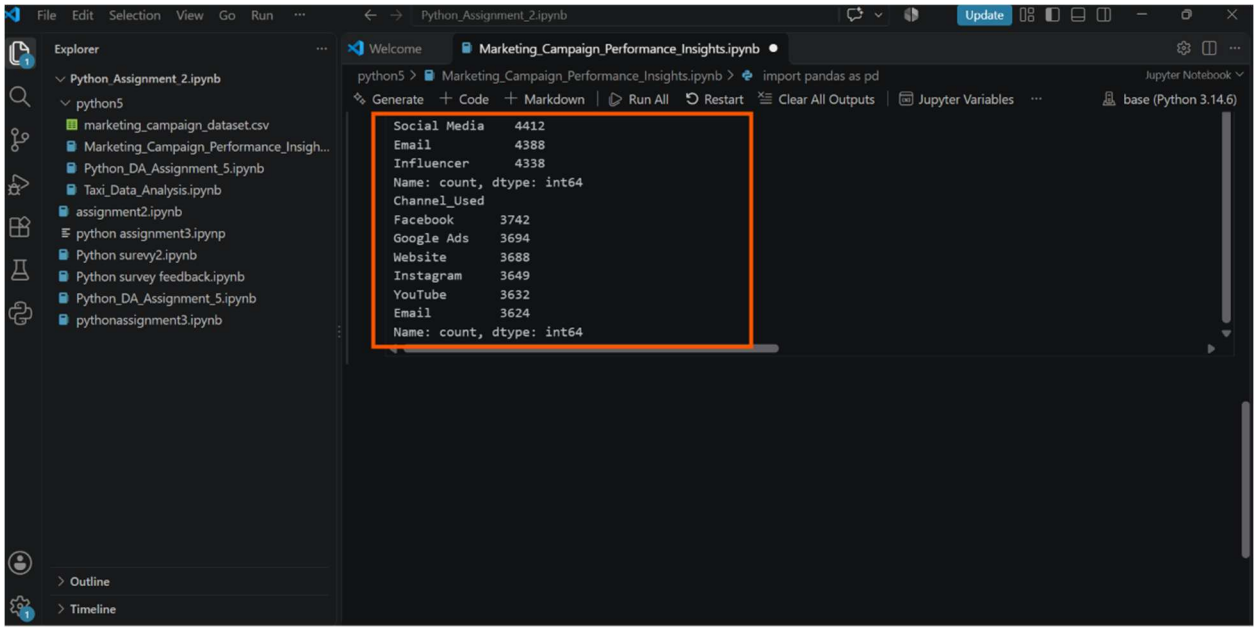
Task 8: Count campaign types

print(df["Campaign_Type"].value_counts())



Task 9: Count channels used

print(df["Channel_Used"].value_counts())

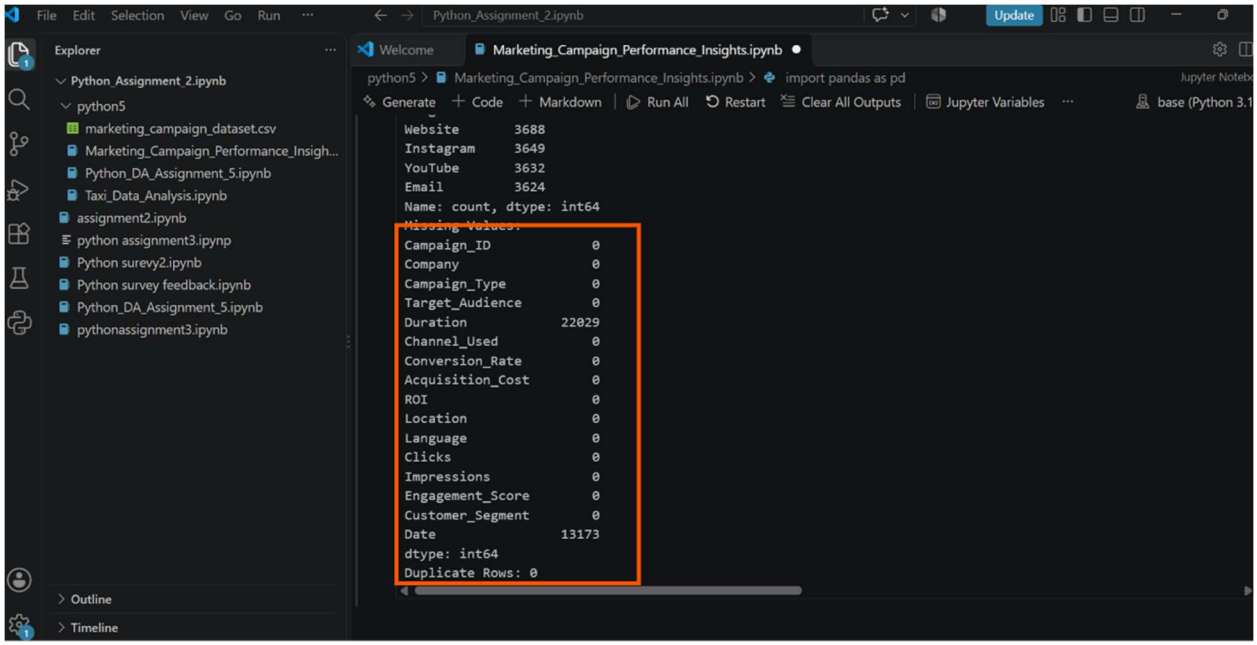


Additional data-quality checks

print("Missing Values:")

print(df.isnull().sum())

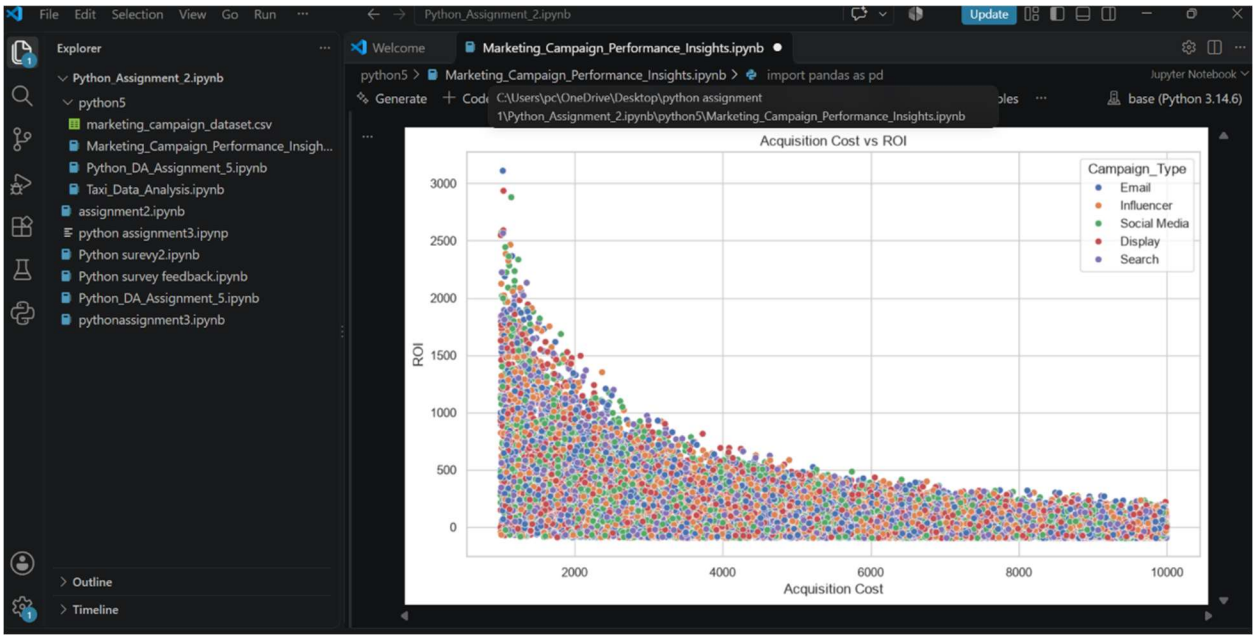
print("Duplicate Rows:", df.duplicated().sum())



Part 2: Exploratory Data Analysis A. Campaign Performance Analysis

1. Scatter Plot: Acquisition Cost vs ROI **Objective:** Examine whether acquisition cost is associated with campaign profitability.

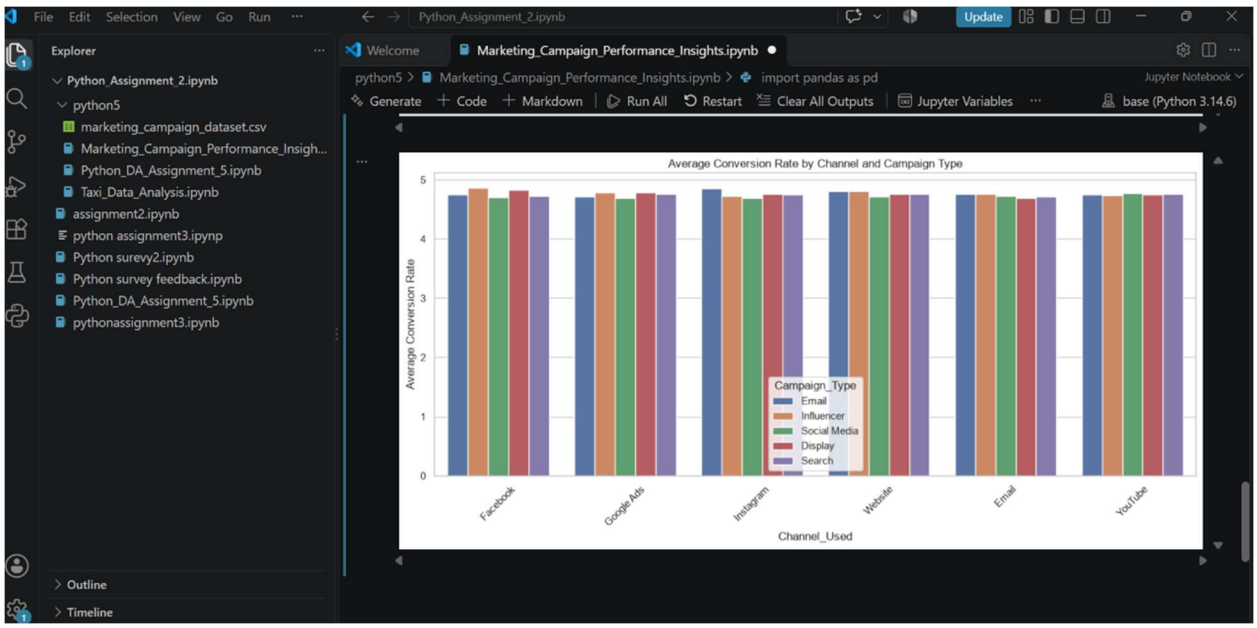
```
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x="Acquisition_Cost",
    y="ROI",
    hue="Campaign_Type"
)
plt.title("Acquisition Cost vs ROI")
plt.xlabel("Acquisition Cost")
plt.ylabel("ROI")
plt.tight_layout()
plt.show()
```



Interpretation: Examine whether higher acquisition costs are associated with higher or lower ROI. The actual relationship must be determined from the chart.

2. Bar Chart: Average Conversion Rate by Channel and Campaign Type

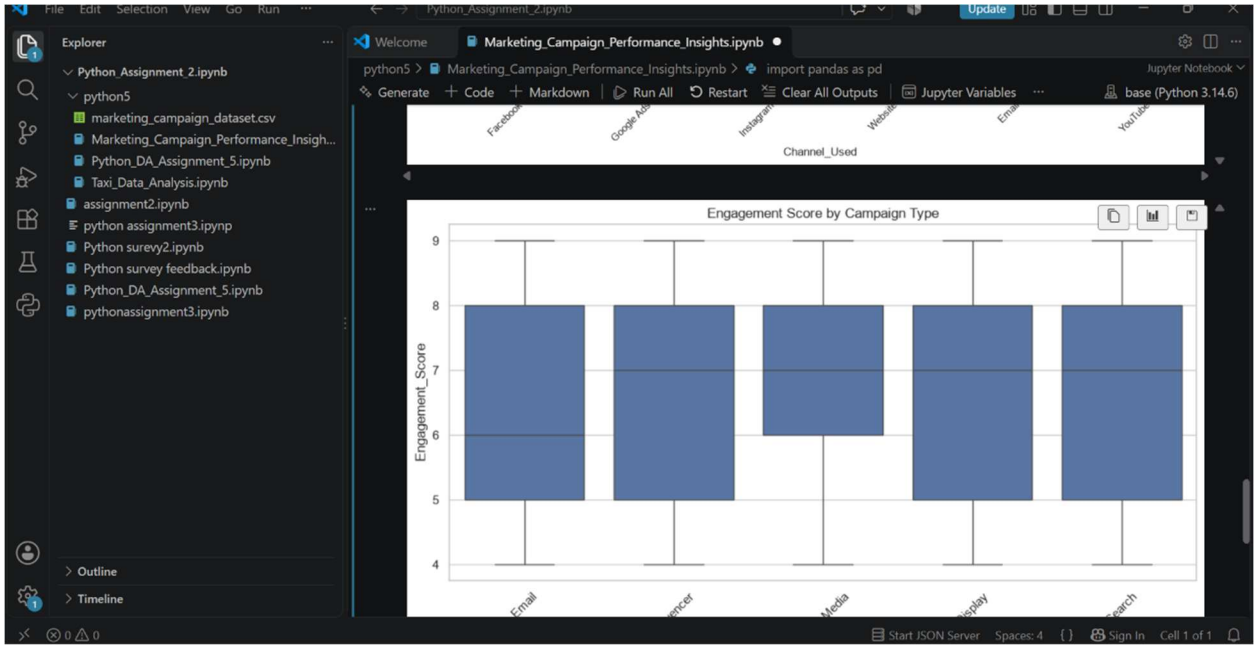
```
plt.figure(figsize=(12, 6))
sns.barplot(
    data=df,
    x="Channel_Used",
    y="Conversion_Rate",
    hue="Campaign_Type",
    estimator="mean",
    errorbar=None
)
plt.title("Average Conversion Rate by Channel and Campaign Type")
plt.xticks(rotation=45)
plt.ylabel("Average Conversion Rate")
plt.tight_layout()
plt.show()
```



Interpretation: Identify which channel and campaign-type combinations achieve higher average conversion rates.

3. Box Plot: Engagement Score by Campaign Type

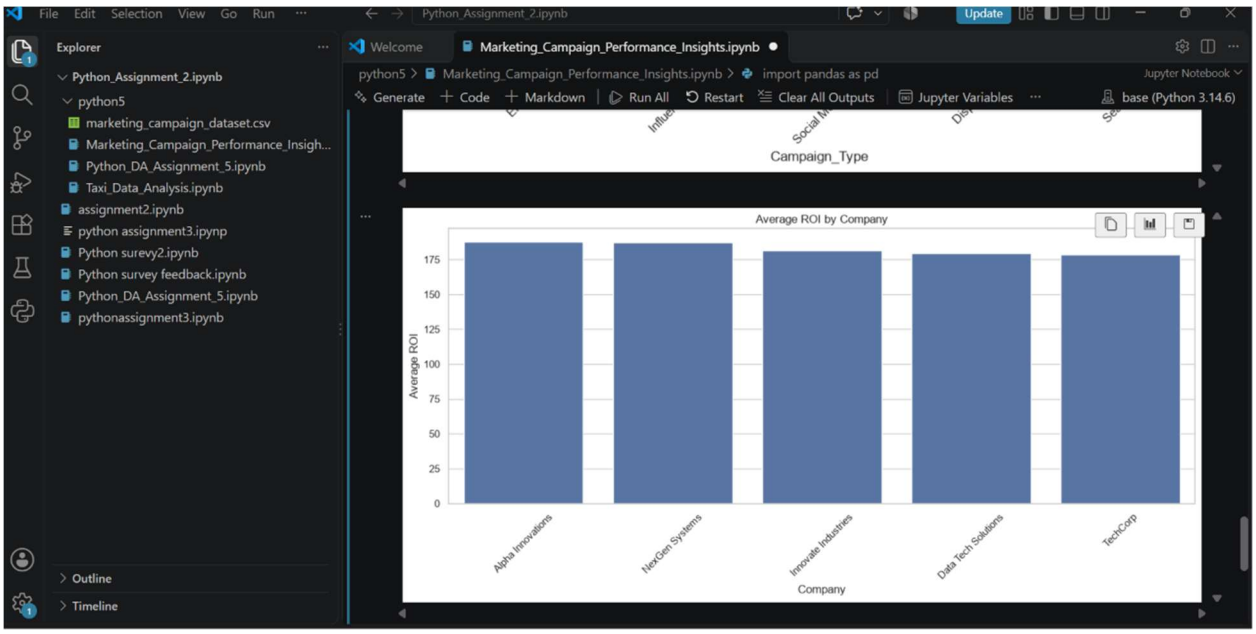
```
plt.figure(figsize=(10, 6))
sns.boxplot(
    data=df,
    x="Campaign_Type",
    y="Engagement_Score"
)
plt.title("Engagement Score by Campaign Type")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Interpretation: Compare median engagement scores, variability, and outliers across campaign types.

4. Bar Chart: Average ROI by Company

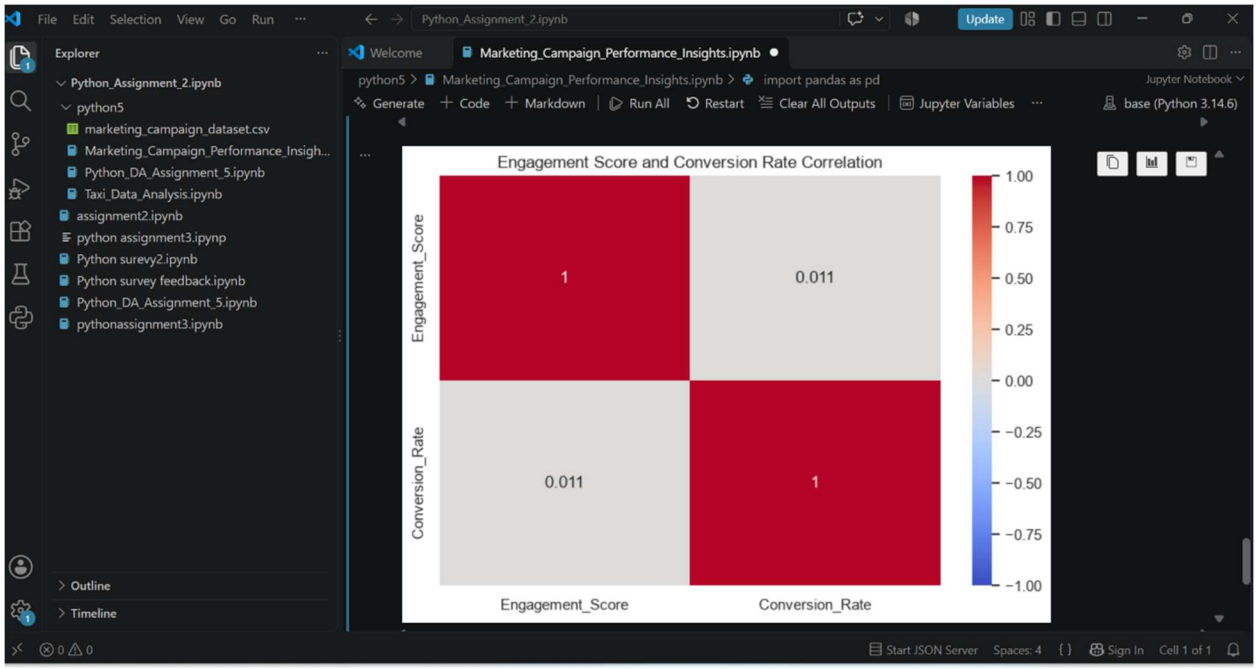
```
roi_company = (
    df.groupby("Company")["ROI"]
    .mean()
    .sort_values(ascending=False))
plt.figure(figsize=(12, 6))
sns.barplot(
    x=roi_company.index,
    y=roi_company.values)
plt.title("Average ROI by Company")
plt.xlabel("Company")
plt.ylabel("Average ROI")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Interpretation: Identify companies with the highest average campaign ROI.

5. Heatmap: Engagement Score vs Conversion Rate

```
correlation = df[
    ["Engagement_Score", "Conversion_Rate"]].corr()
plt.figure(figsize=(7, 5))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    vmin=-1,
    vmax=1)
plt.title("Engagement Score and Conversion Rate Correlation")
plt.tight_layout()
plt.show()
```

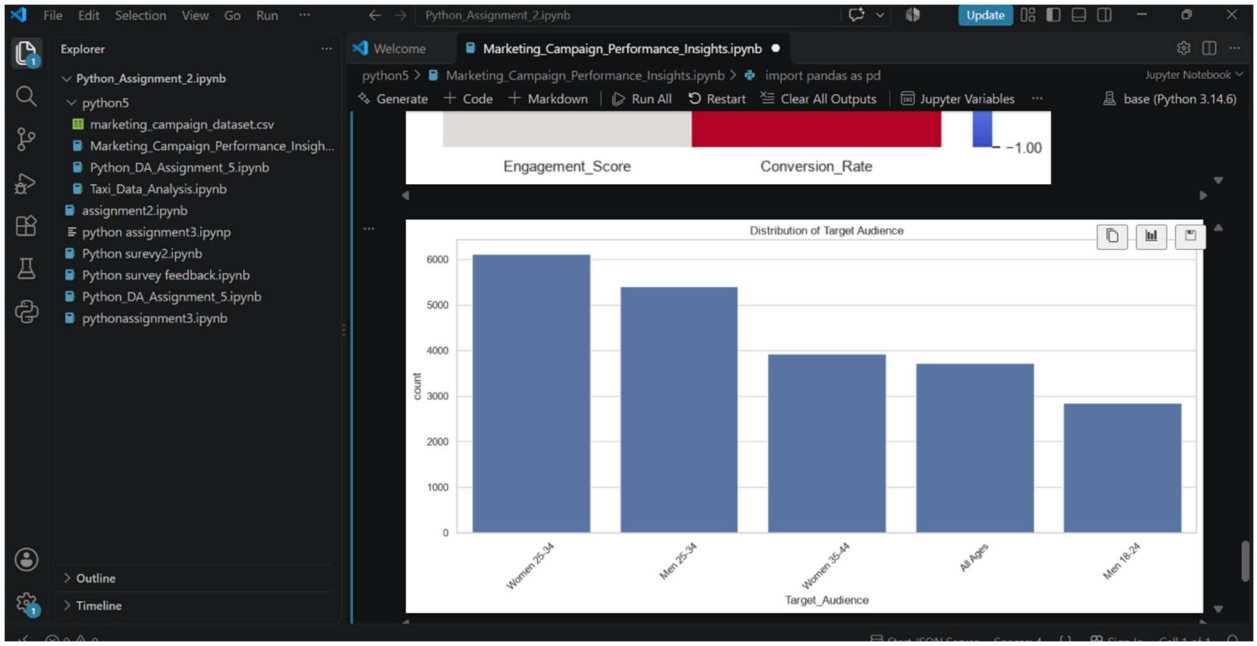


Interpretation: A correlation close to +1 indicates a strong positive linear relationship, while a value near zero indicates little linear relationship.

B. Customer Segmentation Analysis

6. Count Plot: Target Audience Distribution

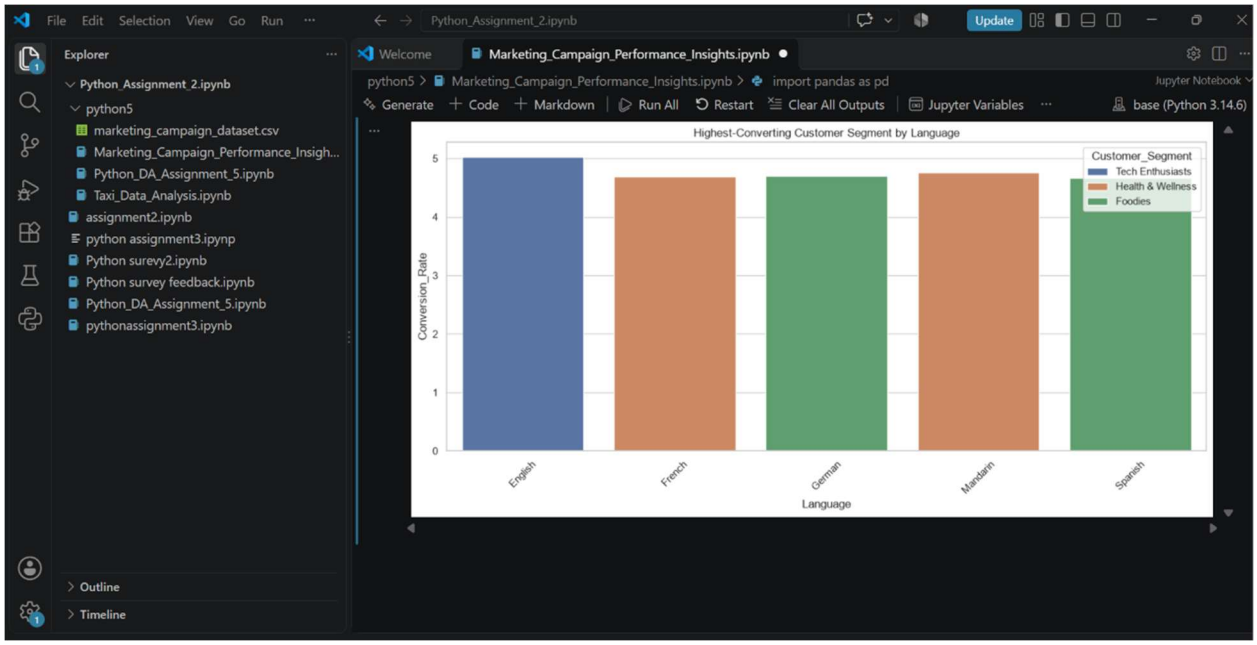
```
plt.figure(figsize=(12, 6))
sns.countplot(
    data=df,
    x="Target_Audience",
    order=df["Target_Audience"].value_counts().index)
plt.title("Distribution of Target Audience")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Interpretation: Identify which target audiences appear most frequently in the dataset.

7. Bar Chart: Highest Conversion Rate by Customer Segment and Language

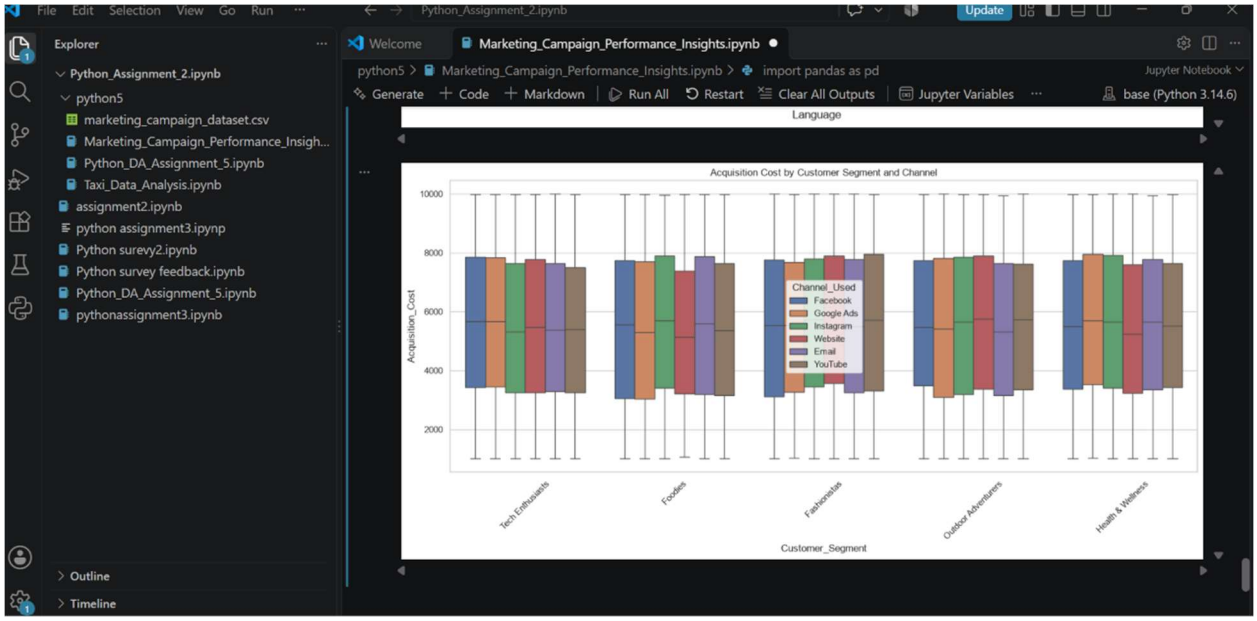
```
segment_conversion = (df.groupby(["Language", "Customer_Segment"])["Conversion_Rate"]
   .mean() .reset_index())
highest_segments = segment_conversion.loc[
   segment_conversion.groupby("Language")["Conversion_Rate"].idxmax()]
plt.figure(figsize=(12, 6))
sns.barplot(data=highest_segments,x="Language",y="Conversion_Rate",
   hue="Customer_Segment")
plt.title("Highest-Converting Customer Segment by Language")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Interpretation: Determine which customer segment has the highest average conversion rate within each language.

8. Box Plot: Acquisition Cost by Customer Segment and Channel

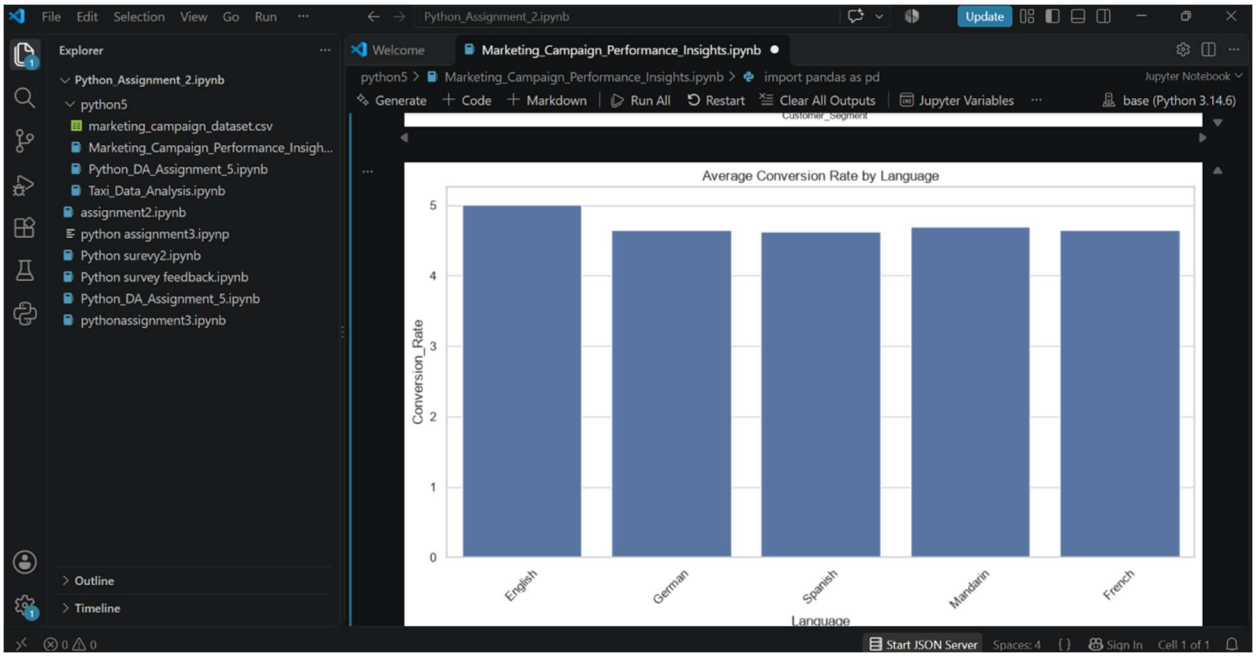
```
plt.figure(figsize=(14, 7))
sns.boxplot(
    data=df,
    x="Customer_Segment",
    y="Acquisition_Cost",
    hue="Channel_Used")
plt.title("Acquisition Cost by Customer Segment and Channel")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Interpretation: Compare acquisition costs across customer segments and channels.

9. Bar Chart: Average Conversion Rate by Language

```
plt.figure(figsize=(10, 6))
sns.barplot(
    data=df,
    x="Language",
    y="Conversion_Rate",
    estimator="mean",
    errorbar=None)
plt.title("Average Conversion Rate by Language")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

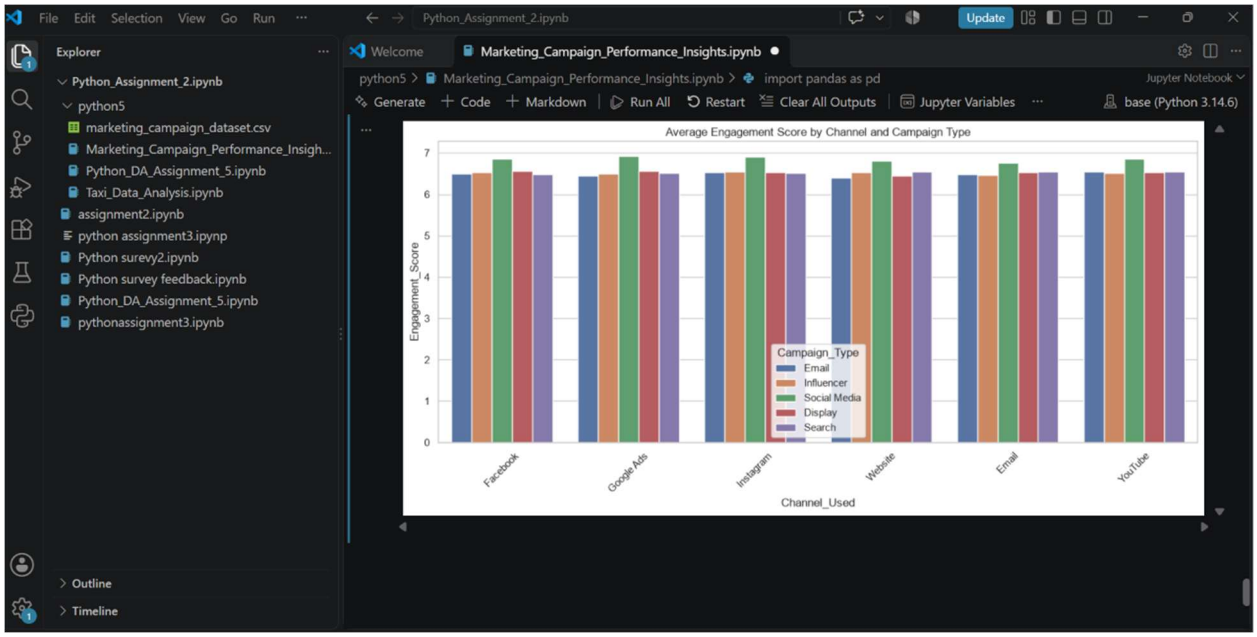


Interpretation: Identify languages associated with higher average conversion rates.

C. Channel Effectiveness Analysis

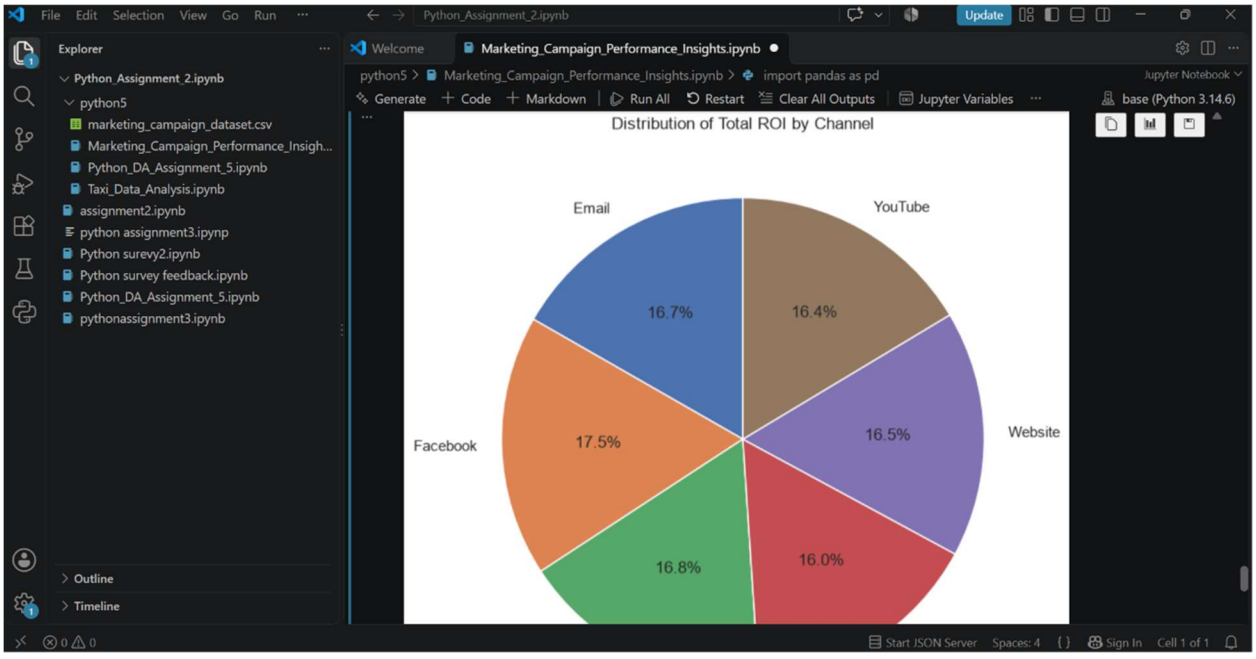
10. Bar Chart: Engagement Score by Channel and Campaign Type

```
plt.figure(figsize=(12, 6))
sns.barplot(
    data=df,
    x="Channel_Used",
    y="Engagement_Score",
    hue="Campaign_Type",
    errorbar=None)
plt.title("Average Engagement Score by Channel and Campaign Type")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



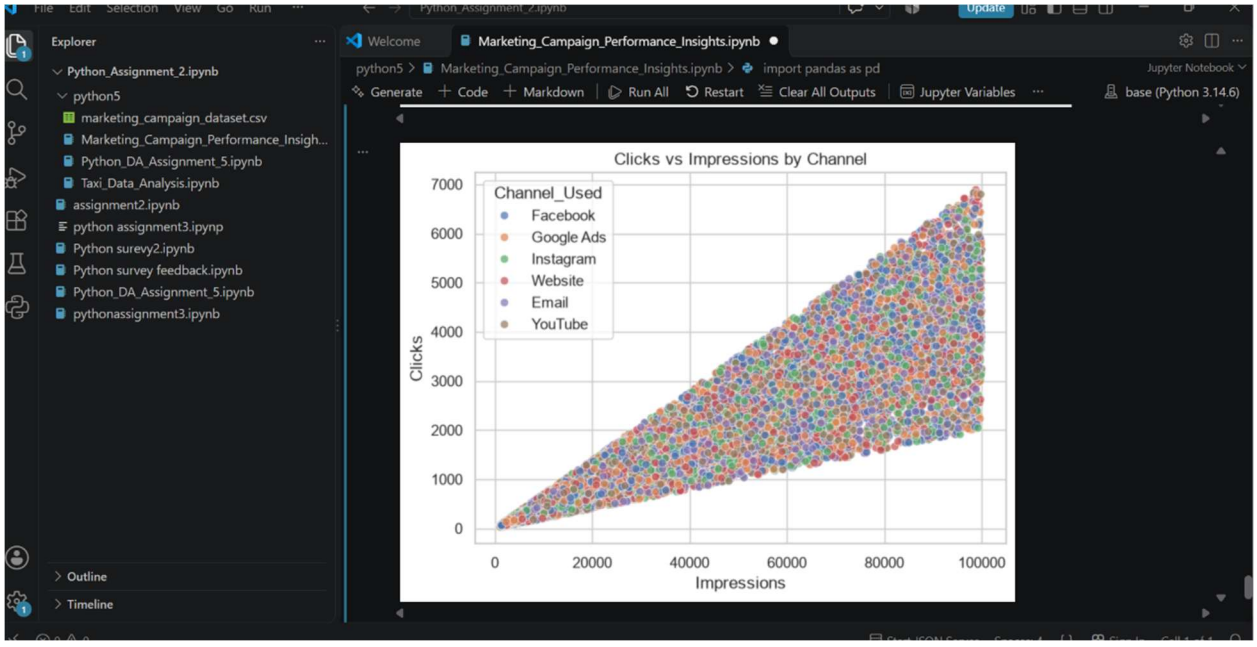
11. Pie Chart: Total ROI by Channel

```
roi_channel = df.groupby("Channel_Used")["ROI"].sum()
if (roi_channel >= 0).all() and roi_channel.sum() > 0:
    plt.figure(figsize=(8, 8))
plt.pie(
        roi_channel,
        labels=roi_channel.index,
        autopct="%1.1f%%",
        startangle=90 )
plt.title("Distribution of Total ROI by Channel")
    plt.show()
else:
    print("Pie chart is not suitable because ROI totals include negative values or have no positive total.")
```



12. Scatter Plot: Clicks vs Impressions by Channel

```
plt.figure(figsize=(12, 6))
sns.scatterplot(
    data=df,
    x="Impressions",
    y="Clicks",
    hue="Channel_Used",
    alpha=0.7
)
plt.title("Clicks vs Impressions by Channel")
plt.xlabel("Impressions")
plt.ylabel("Clicks")
plt.tight_layout()
plt.show()
```

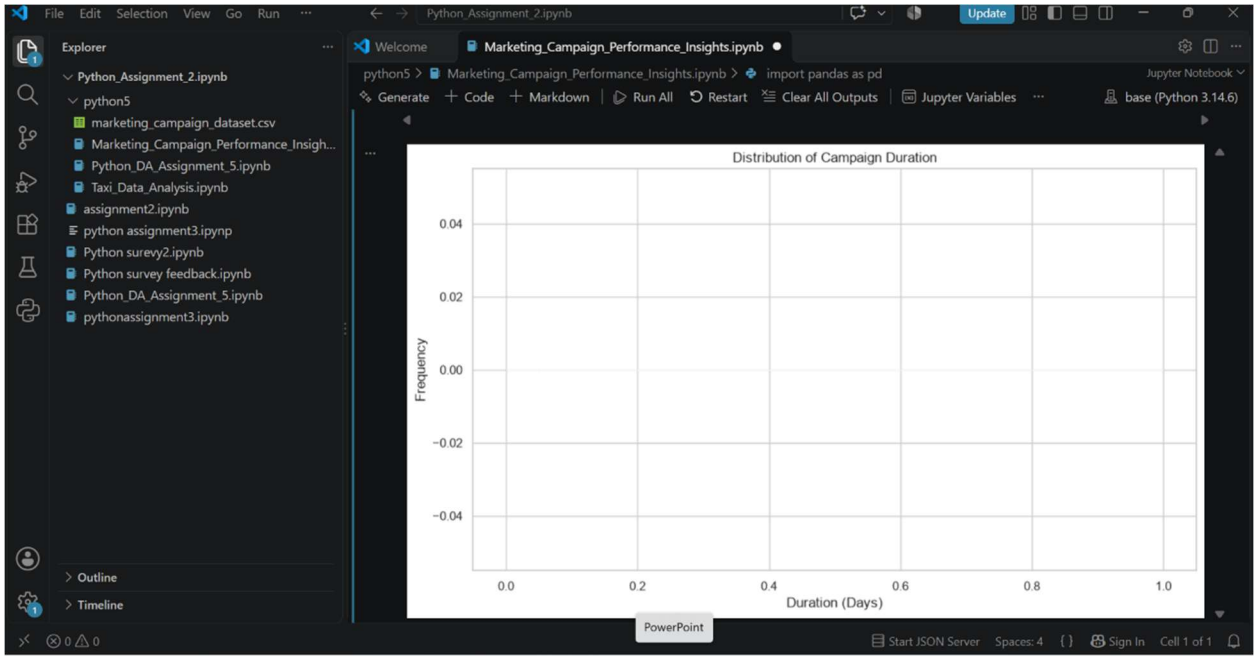


Interpretation: Examine how clicks vary with impressions across marketing channels.

D. Time-Based Analysis

13. Histogram: Campaign Duration

```
plt.figure(figsize=(10, 6))
sns.histplot(
    data=df,
    x="Duration",
    bins=20,
    kde=True
)
plt.title("Distribution of Campaign Duration")
plt.xlabel("Duration (Days)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



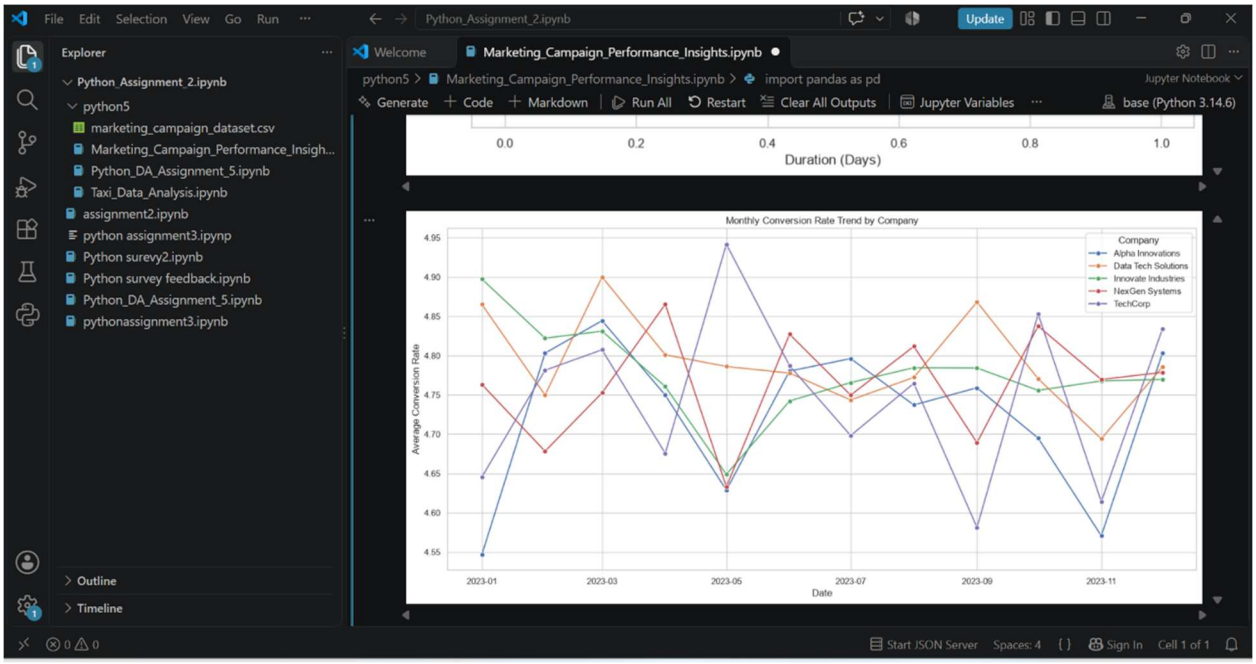
14. Line Chart: Conversion Rate Over Time by Company

```
monthly_conversion = (
    df.dropna(subset=["Date"])
    .groupby([
        pd.Grouper(key="Date", freq="MS"),
        "Company"
    ])["Conversion_Rate"]
    .mean()
    .reset_index()
)

plt.figure(figsize=(14, 7))

sns.lineplot(
    data=monthly_conversion,
    x="Date",
    y="Conversion_Rate",
    hue="Company",
    marker="o"
)

plt.title("Monthly Conversion Rate Trend by Company")
plt.xlabel("Date")
plt.ylabel("Average Conversion Rate")
plt.tight_layout()
plt.show()
```



Interpretation: Identify changes in conversion performance over time for each company.

15. Line Chart: Engagement Score Trend

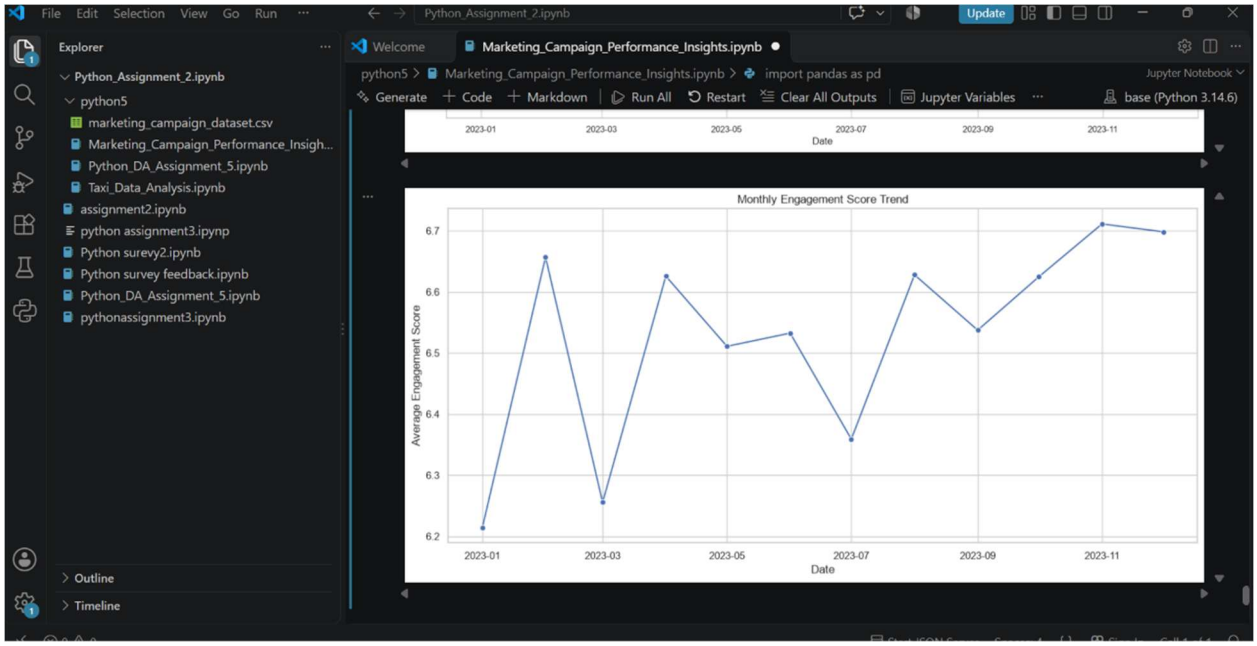
```
monthly_engagement = (
    df.dropna(subset=["Date"])
    .groupby(pd.Grouper(key="Date", freq="MS"))["Engagement_Score"]
    .mean()
    .reset_index()
)


plt.figure(figsize=(12, 6))


sns.lineplot(
    data=monthly_engagement,
    x="Date",
    y="Engagement_Score",
```

```
    marker="o"
)

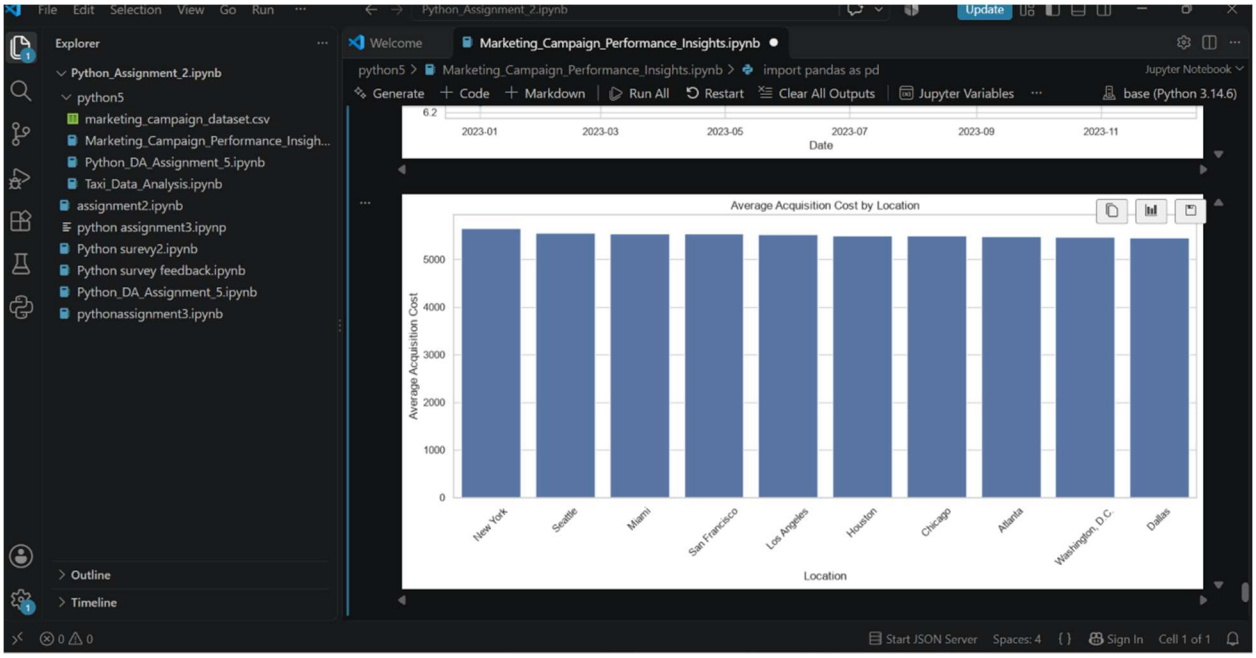
plt.title("Monthly Engagement Score Trend")
plt.xlabel("Date")
plt.ylabel("Average Engagement Score")
plt.tight_layout()
plt.show()
```



E. Geographic Analysis

16. Bar Chart: Acquisition Cost by Location

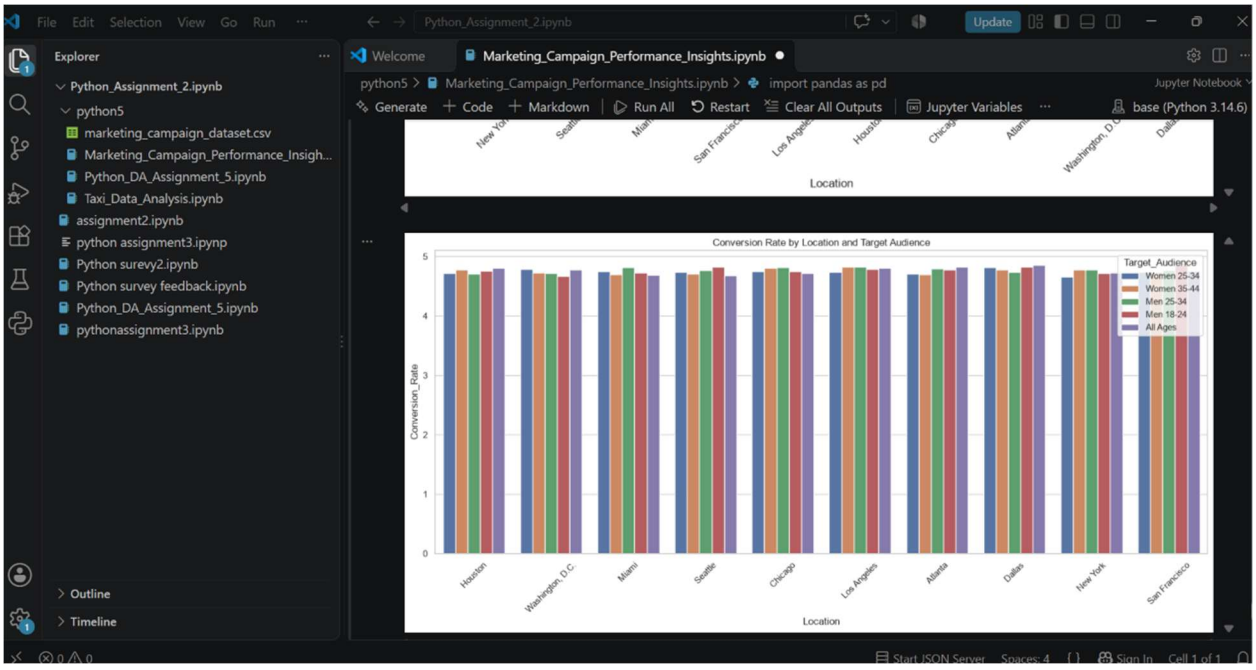
```
location_cost = (df.groupby("Location")["Acquisition_Cost"]
    .mean().sort_values(ascending=False))
plt.figure(figsize=(12, 6))
sns.barplot(
    x=location_cost.index,
    y=location_cost.values)
plt.title("Average Acquisition Cost by Location")
plt.xticks(rotation=45)
plt.ylabel("Average Acquisition Cost")
plt.tight_layout()
plt.show()
```



Interpretation: Identify the location with the highest average customer acquisition cost.

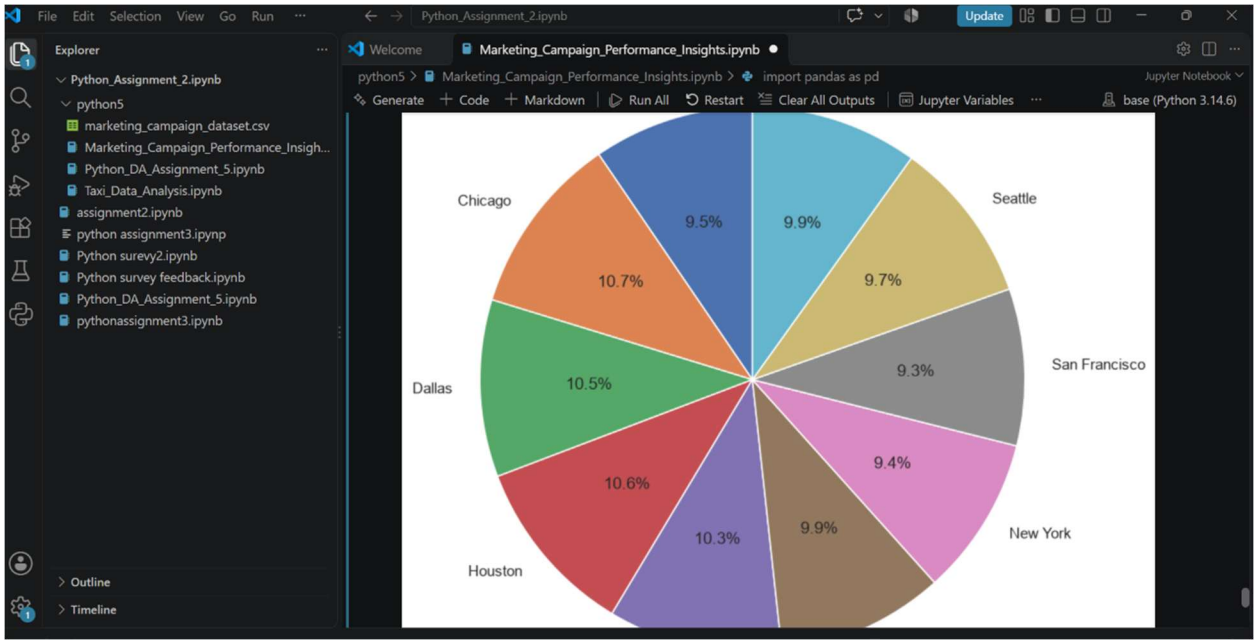
17. Bar Chart: Conversion Rate by Location and Target Audience

```
plt.figure(figsize=(14, 7))
sns.barplot(
    data=df,
    x="Location",
    y="Conversion_Rate",
    hue="Target_Audience",
    errorbar=None)
plt.title("Conversion Rate by Location and Target Audience")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



18. Pie Chart: ROI by Location

```
location_roi = df.groupby("Location")["ROI"].sum()
if (location_roi >= 0).all() and location_roi.sum() > 0:
    plt.figure(figsize=(9, 9))
plt.pie(location_roi,
        labels=location_roi.index,
        autopct="%1.1f%%",
        startangle=90)
plt.title("Distribution of Total ROI by Location")
    plt.show()
else:print("Pie chart is not suitable for these ROI totals.")
```



Part 3: Additional Marketing Performance KPIs

```
print("Total Campaigns:", df["Campaign_ID"].nunique())

print("Average Conversion Rate:",
    round(df["Conversion_Rate"].mean(), 2))

print("Average ROI:",
    round(df["ROI"].mean(), 2))

print("Average Acquisition Cost:",
    round(df["Acquisition_Cost"].mean(), 2))

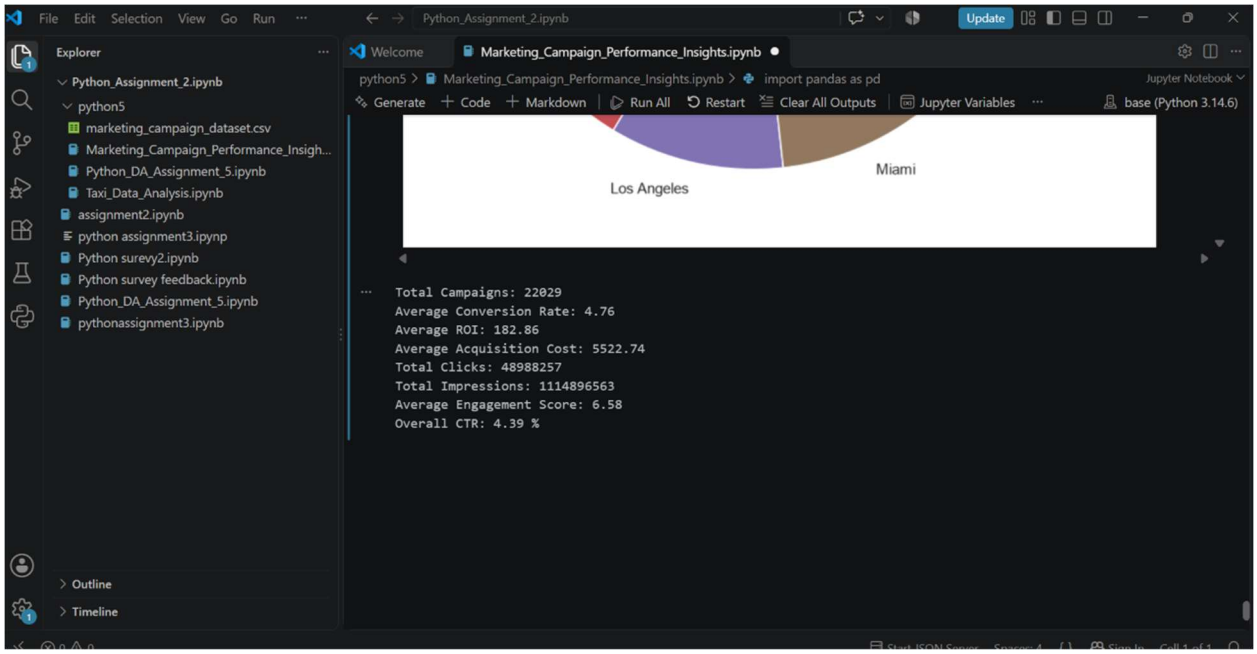
print("Total Clicks:", df["Clicks"].sum())

print("Total Impressions:", df["Impressions"].sum())

print("Average Engagement Score:",
    round(df["Engagement_Score"].mean(), 2))

total_impressions = df["Impressions"].sum()

if total_impressions > 0:
   ctr = df["Clicks"].sum() / total_impressions * 100
   print("Overall CTR:", round(ctr, 2), "%")
```



KPI	Formula	Purpose
Total Campaigns	Count of unique Campaign_ID	Measures campaign volume
Average Conversion Rate	Mean Conversion_Rate	Evaluates campaign effectiveness
Average ROI	Mean ROI	Compares profitability
Average Acquisition Cost	Mean Acquisition_Cost	Measures acquisition efficiency
Total Clicks	Sum of Clicks	Measures audience interaction
Total Impressions	Sum of Impressions	Measures campaign reach
Click-Through Rate	Clicks / Impressions × 100	Measures click performance
Average Engagement Score	Mean Engagement_Score	Evaluates audience engagement

Part 4: Insights and Conclusions

After executing all the visualizations, I prepared My analysis under four categories.

Analysis Type	What to Examine
Descriptive Analysis	Campaign counts, average ROI, conversion rates, acquisition costs, and engagement scores
Diagnostic Analysis	Differences in performance across campaign types, channels, companies, and customer segments
Predictive Analysis	Potential future trends based on historical campaign performance; forecasting requires additional modeling
Prescriptive Analysis	Recommendations for budget allocation, audience targeting, and channel optimization

Recommended Business Actions

1. **Prioritize channels with consistently strong ROI and conversion rates.**
2. **Investigate campaigns with high acquisition costs and low profitability.**
3. **Optimize audience targeting based on customer-segment performance.**
4. **Compare geographic markets to identify opportunities for more efficient spending.**
5. **Monitor monthly conversion and engagement trends to improve campaign planning.**

These are proposed actions, not findings confirmed by the dataset.

Part 5: Project Conclusion

Conclusion

The Marketing Campaign Performance Insights project demonstrates the application of Python for analyzing and visualizing digital marketing campaign data.

Using Pandas, NumPy, Matplotlib, and Seaborn, the project examines campaign performance through key metrics such as conversion rate, customer acquisition cost, return on investment (ROI), clicks, impressions, and engagement scores.

The analysis provides a framework for comparing campaign types, marketing channels, customer segments, companies, languages, and geographic locations. Time-based visualizations also support the identification of changes in campaign performance.

The project highlights how exploratory data analysis can help marketing teams evaluate campaign effectiveness, identify potential optimization opportunities, and make informed decisions about future marketing strategies.

Overall, Python provides an effective approach to transforming marketing campaign data into visual insights that can support data-driven decision-making.